MIT 8103 Advanced Database Systems: Portfolio Summary

Name: Goodness Etim | Student ID: 301914284
Case study: Bank / Fintech Platform

https://github.com/Kingee1/Etim_Goodness_301914284

Case study

This fictional fintech platform manages customers, single-owner naira accounts, deposits, withdrawals, internal transfers and transaction logs. Customers request transactions; operations staff maintain records; reversal officers process corrections; auditors review activity. Loans, fees, cards, external payments, foreign exchange and authentication are excluded. PostgreSQL and MongoDB run locally in the same Codespace. The five activities follow the assessment brief [1].

Portfolio 1: Database Design and Modelling

The relational design contains customer, account, staff, transaction and transaction_log. Identity primary keys identify records; foreign keys connect them. Unique fields identify customers, accounts, staff and transactions. Checks enforce lowercase emails, NGN currency, positive amounts, permitted statuses and valid debit/credit combinations. A unique original_transaction_id permits one reversal per original transaction.

The normalisation explanation separates customer, account, staff, transaction and log details to reduce repeated data. The initial valid dataset contains 2 customers, 3 accounts, 3 staff, 7 transactions and 7 logs. Invalid inserts demonstrate required-field, uniqueness, foreign-key and CHECK restrictions; final counts remain unchanged. An invalid transaction type fails the account-shape check, so it does not independently demonstrate the separate type check.

The ER diagram expresses the intended model. Automatic funds checks, date checks against account opening, logging, authorisation and protection of completed records still require additional workflow logic. They are not all enforced by the schema.

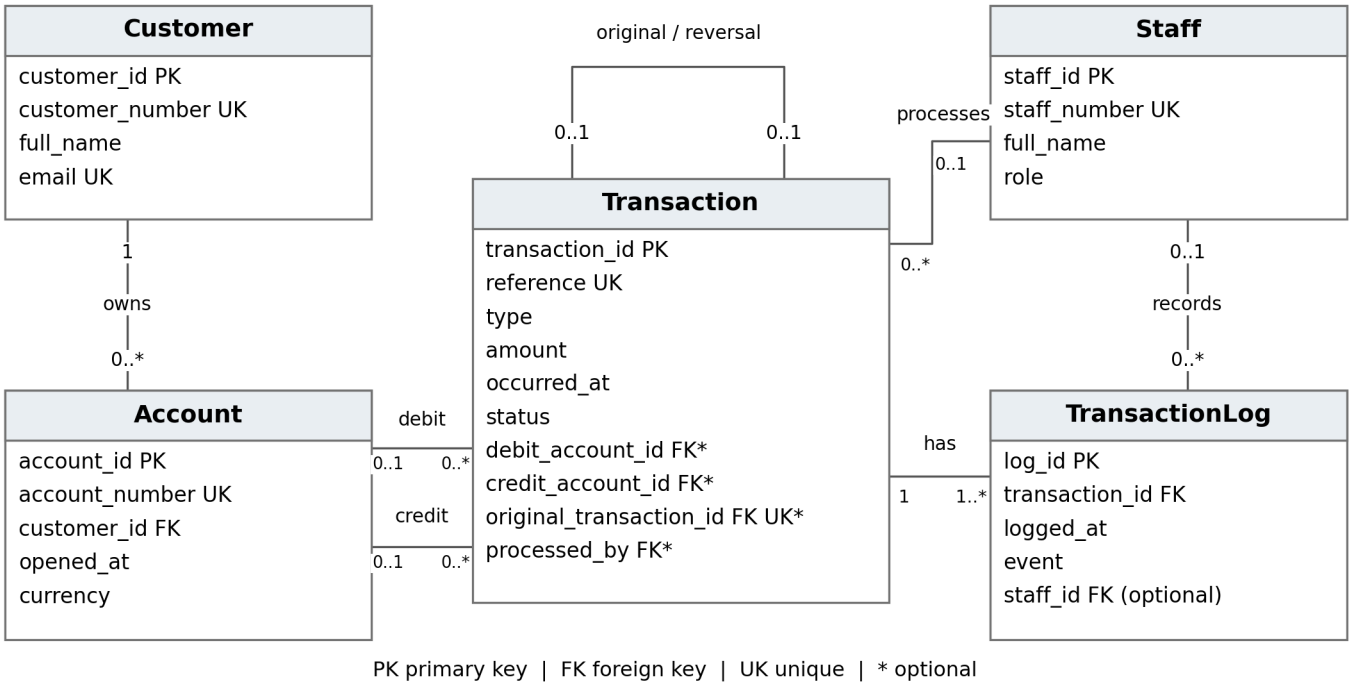


Figure 1. ER model from Portfolio 1; field types are omitted for readability. 1 means exactly one; 0..1 means optional; 0..* means zero or many; 1..* means one or many.

Portfolio 2: Query Processing and Optimisation

Three queries initially return pending transfer DEMO-T005 for ₦100 from Ada James to Ben Cole, unused account DEMO-A003, and transaction counts of 4, 5 and 0 for accounts DEMO-A001, DEMO-A002 and DEMO-A003. LEFT JOIN retains the zero-count account. These results describe the state before Portfolio 3 changes DEMO-T005 to failed.

A separate p2_benchmark.transaction_history table contains 100,000 synthetic deposits, one minute apart. The unchanged query counts 60 deposits between 09:00 inclusive and 10:00 exclusive on 2 October 2026 (+01). Each phase uses one warm-up and three measured runs. A B-tree index on occurred_at changes the observed plan [2].

Measurement	Before index	After index
Scan type	Seq Scan	Index Only Scan
Execution times (ms)	12.164, 7.642, 7.526	0.070, 0.072, 0.063
Median time (ms)	7.642	0.070
Shared buffer hits (third run)	1344	4

Before indexing, the scan checks 100,000 rows and rejects 99,940. Afterwards, the index-only scan finds 60 entries with zero heap fetches. Both phases use cached data; Codespaces timings vary. This is a narrow synthetic benchmark, not production performance. The index also consumes storage and adds write-maintenance work.

Portfolio 3: Transactions and Concurrency

COMMIT saves DEMO-T005 as failed together with its new log. Repeating the guarded update affects zero rows. A second transaction inserts P3-ROLLBACK, then a zero amount triggers transaction_amount_positive. ROLLBACK removes that insert; the earlier committed change and its single new log remain [3].

Two READ COMMITTED sessions attempt to reverse completed deposit DEMO-T002. A inserts P3-REV-A and its log without committing. B sees zero reversals, then waits on its competing insert; the activity check reports Lock. After A commits, B receives transaction_original_transaction_id_key and rolls back. Exactly one ₦5,000 reversal remains; P3-REV-B is absent. The unique constraint prevents the conflict; isolation alone does not enforce every banking rule [3].

Portfolio 4: NoSQL and Advanced Data Models

MongoDB 8.0.34 stores four fictional documents in fintech_events.transaction_events, separately from SQL. Embedded customer, account and transaction details accompany recorded or reversed events. Account DEMO-A002 history returns E001, E003, E004 and E002 in time order; the nested credit-reversal filter returns only E004. Dates use BSON dates and amounts use Decimal128. Embedding supports event-history reads with varying details, but copied data is a snapshot. No validator, SQL synchronisation or complete balance ledger is implemented [4].

Portfolio 5: Distributed and Cloud Database Exercise

This activity is a discussion, not a deployed replica set. The proposal places three voting, data-holding MongoDB members on separate hosts and failure zones. A primary accepts writes; two secondaries copy events such as E004 asynchronously. A reachable majority can elect a new primary after failure, with a write interruption. Majority acknowledgement requires two members in this design; secondary reads can lag. Four documents do not justify sharding [5].

A managed deployment requires authentication, encryption, restricted access, cost review and backups tested by restoration. Replication cannot replace backups, protect PostgreSQL automatically or synchronise the two databases. No replication, sharding or failover test is claimed [5].

Evidence

Full scripts, diagrams, results and existing screenshots are linked from each portfolio README: Portfolio 1 | Portfolio 2 | Portfolio 3 | Portfolio 4 | Portfolio 5. Earlier screenshots remain evidence of their original database states. GitHub contains the reproducible scripts, not a backup of the running databases.

Individual reflection

Some parts were easier to follow than others. I followed ChatGPT guidance to calculate balances from completed transactions instead of storing a separate balance, avoiding conflicting values. I sometimes confused the PostgreSQL prompt with the regular terminal and entered commands in the wrong place. I checked the prompt, cleared unfinished input and reran the correct commands. This helped me understand the different tools and why I needed to check each result before continuing.

References

[1] Miva University. MIT 8103 assessment brief. Assessment requirements.

[2] PostgreSQL 16. Using EXPLAIN.

[3] PostgreSQL 16. Transactions; Isolation; Unique-index checks.

[4] MongoDB. Data modelling; insertMany; Nested queries.

[5] MongoDB. Replication; Elections; Write concern; Sharding; Backups.

[6] MIT8103 Codelab. Tutorial structure and practical guidance. Repository.

AI-use declaration

I used ChatGPT extensively to adapt the codelab [6] to my fintech case, propose the design, generate SQL and MongoDB scripts, explain results, troubleshoot errors and draft documentation, including this summary. I entered the commands myself, shared outputs for review, corrected input and file issues, and checked results and GitHub links with guidance. ChatGPT also helped phrase my reflection using difficulties recorded during the work. I remain responsible for reviewing, understanding and explaining the submitted work.